ResearchReel Architecture Documentation

Executive Summary

ResearchReel is a microservices-based, cloud-native platform designed for sharing research via short-form videos (Reels) and interactive research documents. The platform enables researchers, students, scholars, professors, moderators, and administrators to transform academic content into engaging multimedia formats while maintaining research integrity and fostering collaboration.

System Architecture Overview

High-Level Components

1. **Frontend Web Application** (`/frontend`)
    - Next.js 16.2.2 React framework
    - TypeScript for type safety
    - Tailwind CSS for styling
    - SWR or React Query for data fetching
    - Socket.IO client for real-time features
2. **API Gateway & Backend** (`/backend`)
    - Node.js with Express.js framework
    - RESTful API design
    - WebSocket support via Socket.IO
    - PostgreSQL as primary database

    - Redis for caching and BullMQ job queuing
    - Modular architecture with separate route handlers

3. **Python RAG Microservice** (`/backend/research_rag`)
    - FastAPI framework
    - Google Gemini AI for natural language processing
    - Qdrant vector database for semantic search
    - Document processing pipeline (PDF, DOCX, TXT)
    - Text chunking and embedding generation

4. **Background Task Worker** (`video-worker`)
    - BullMQ for job queue management
    - FFmpeg for video processing
    - Integration with RAG service for content generation
    - TTS (Text-to-Speech) integration for audio generation

5. **Infrastructure Services**
    - PostgreSQL 15 (primary database)
    - Redis 7 (cache and message broker)
    - Optional: Qdrant (vector search)
    - Optional: AWS S3 (object storage)

Deployment Architecture

Development Environment

- Docker Compose orchestration
- All services running in separate containers
- Local development with hot reloading
- Environment-specific configuration via .env files

Production Environment

- Kubernetes/EKS orchestration
- Helm charts for service deployment

- Ingress controller (NGINX) for external access
- Auto-scaling groups for handling variable load
- CDN for static asset delivery
- Managed services for databases (AWS RDS, Elasticache)

Detailed Component Architecture

Frontend Architecture

Technology Stack

- **Framework**: Next.js 16.2.2 (React 18)
- **Language**: TypeScript
- **Styling**: Tailwind CSS
- **State Management**: React Context API + SWR
- **Real-time Communication**: Socket.IO client
- **UI Components**: Custom component library with Lucide icons
- **Document Handling**: React-PDF for PDF viewing/rendering
- **3D Visualization**: Three.js with @react-three/fiber
- **Drag & Drop**: @hello-pangea/dnd for Kanban boards
- **Animations**: Framer Motion, GSAP
- **Analytics**: PostHog
- **Error Tracking**: Sentry

Key Features Implementation

1. **Authentication Flow**
    - JWT-based auth with HttpOnly cookies
    - OTP verification for registration
    - Protected routes via Higher-Order Components
    - Automatic token refresh mechanism
2. **Content Creation Interface**
    - Rich text editor for posts

    - Media upload with drag-and-drop
    - DOI lookup integration for academic metadata extraction
    - Reel generator with prompt controls
    - Project workspace with Kanban board

3. **Real-time Features**
    - Live feed updates via WebSocket
    - Real-time notifications
    - Collaborative editing indicators
    - Live cursor positions in shared documents

4. **Accessibility & Performance**
    - WCAG 2.1 AA compliance
    - Server-side rendering for SEO
    - Image optimization with Next.js Image
    - Code splitting and lazy loading
    - Critical CSS inlining
    - Service worker for offline capabilities

Backend Architecture

API Gateway Service

- **Entry Point**: Single entry point for all client requests
- **Routing**: RESTful API endpoints organized by resource
- **Middleware Stack**:
    - Helmet for security headers
    - CORS configuration
    - Request logging (Morgan)
    - Body parsing (JSON, multipart/form-data)
    - Rate limiting (express-rate-limit)
    - Authentication verification
    - Request validation (Zod schemas)

Core Services

1. **Authentication Service**
    - JWT token generation and validation
    - Password hashing with Argon2
    - OTP generation and verification (via Redis)
    - Role-based access control (RBAC)
    - Session management
2. **Content Management Service**
    - Post creation, retrieval, updating, deletion
    - Media file handling and validation
    - DOI resolution and metadata extraction
    - Content moderation workflow
    - Search integration
3. **Reel Generation Service**
    - Video generation job queuing
    - Integration with FFmpeg processing
    - Status tracking and progress reporting
    - Thumbnail extraction and optimization
    - HLS streaming preparation
4. **Project & Collaboration Service**
    - Project lifecycle management
    - Kanban board implementation
    - Task assignment and tracking
    - Version control for documents
    - Collaborator invitation system
5. **Messaging Service**
    - Real-time chat functionality
    - Conversation management
    - Message persistence and retrieval
    - Read receipts and typing indicators
    - File upload within messages
6. **AI Assistance Service**

    - RAG (Retrieval-Augmented Generation) pipeline
    - Context-aware AI responses
    - Document summarization
    - Question answering based on uploaded research
    - Content generation assistance

Data Layer

- **Primary Database**: PostgreSQL 15
    - Connection pooling for efficient resource utilization
    - Read replicas for scaling read operations
    - Regular automated backups
    - Migration system with node-pg-migrate
    - Indexing strategy for query performance
- **Cache Layer**: Redis 7
    - Session storage and OTP codes
    - Frequently accessed data caching
    - Pub/Sub for real-time notifications
    - BullMQ job queue backend
    - Distributed locking mechanisms
- **Vector Database**: Qdrant (optional but recommended)
    - Embedding storage for semantic search
    - Similarity search for research discovery
    - Hybrid search capabilities (keyword + vector)
    - Scalable horizontal partitioning
- **Object Storage**: AWS S3 (optional but recommended for production)
    - Media file storage (images, videos, documents)
    - CDN integration for global delivery
    - Lifecycle policies for cost optimization
    - Versioning for backup and recovery

RAG Microservice Architecture

Overview

The Retrieval-Augmented Generation (RAG) microservice is responsible for enhancing AI capabilities with domain-specific knowledge from user-uploaded research documents.

Core Components

1. **Document Ingestion Pipeline**
    - File format support: PDF, DOCX, TXT, MD
    - Text extraction and cleaning
    - Language detection and processing
    - Metadata extraction (authors, dates, DOI, etc.)
2. **Text Processing Module**
    - Sentence-level tokenization
    - Semantic chunking strategies
    - Embedding generation using Gemini
    - Chunk overlap for context preservation
3. **Vector Storage & Retrieval**
    - Qdrant collection management
    - Similarity search with configurable thresholds
    - Hybrid search (vector + keyword filtering)
    - Result re-ranking and diversity enhancement
4. **Generation Engine**
    - Gemini API integration
    - Prompt engineering for research domain
    - Context injection from retrieved documents
    - Temperature and token limit configuration
    - Safety filtering and bias mitigation
5. **Evaluation & Feedback System**
    - Response quality metrics
    - User feedback collection
    - A/B testing framework

    - Continuous improvement loop

Workflow Flow

1. User uploads research document
2. Document processed and stored in object storage
3. Text extracted and cleaned
4. Document chunked into semantic units
5. Embeddings generated for each chunk
6. Chunks stored in vector database with metadata
7. When user queries, relevant chunks retrieved
8. Retrieved context + query sent to Gemini
9. Generated response returned to user

Infrastructure & Deployment

Containerization Strategy

- **Docker Multi-stage Builds**
    - Separate build and runtime stages
    - Minimal base images (Alpine/Distroless)
    - Security scanning in CI pipeline
    - Layer caching for faster builds

Orchestration

- **Development**: Docker Compose
    - Service definitions with health checks
    - Volume mounting for development
    - Environment-specific overrides
    - Network isolation between services
- **Production**: Kubernetes/EKS
    - Helm charts for each service
    - Resource requests and limits

    - Horizontal Pod Autoscaler (HPA)
    - Pod Disruption Budgets (PDB)
    - Node affinity and anti-affinity rules
    - Tolerations and taints for specialized workloads

Networking & Security

- **Service Mesh**: Istio/Linkerd (optional for advanced traffic management)
- **API Gateway**: Kong or AWS API Gateway (edge)
- **Service-to-Service**: Internal cluster IP or service mesh
- **External Access**: Ingress controller with TLS termination
- **Security Policies**: Network policies, PodSecurityPolicies/RBAC
- **Secrets Management**: Kubernetes Secrets or AWS Secrets Manager
- **Image Scanning**: Trivy or similar in CI pipeline

Observability

- **Logging**: Structured JSON logging (Winston for Node, standard logging for Python)
    - Centralized aggregation (ELK stack or similar)
    - Correlation IDs for request tracing
    - Different log levels (debug, info, warn, error)
    - Audit trails for security-relevant events
- **Metrics**: Prometheus + Grafana
    - Application-level metrics (request latency, error rates)
    - Business metrics (active users, content uploads, reel generations)
    - Infrastructure metrics (CPU, memory, disk, network)
    - Custom metrics for domain-specific KPIs
- **Tracing**: Jaeger or AWS X-Ray
    - Distributed tracing across services
    - Span attributes for debugging
    - Latency bottleneck identification
    - Error propagation tracking

CI/CD Pipeline

- **Source Control**: Git with trunk-based development
- **Continuous Integration**:
    - Automated testing on pull requests
    - Code quality checks (ESLint, Prettier)
    - Security scanning (SAST, dependency checks)
    - Docker image building and vulnerability scanning
    - Deployment to staging environment
- **Continuous Deployment**:
    - Blue-green or canary deployments
    - Automated rollback on health check failures
    - Database migration management
    - Feature flag integration
    - Performance benchmarking

Data Models & Relationships

Core Entities

1. **User**
    - id: UUID (PK)
    - email: string (unique)
    - username: string (unique)
    - password_hash: string
    - full_name: string
    - role: enum (guest, user, scholar, professor, moderator, admin)
    - verification_status: enum
    - orcid_id: string (nullable)
    - institution_name: string (nullable)
    - research_interests: string array
    - created_at: timestamp
    - updated_at: timestamp
2. **Post**

- id: UUID (PK)
- author_id: UUID (FK to User)
- content_type: enum (text, paper, dataset, update)
- caption: string
- media_url: string (nullable)
- doi: string (nullable, unique)
- moderation_status: enum (pending, approved, rejected)
- created_at: timestamp
- updated_at: timestamp

3. **Reel**

- id: UUID (PK)
- author_id: UUID (FK to User)
- source_post_id: UUID (FK to Post, nullable)
- title: string
- video_url: string
- hls_url: string (for streaming)
- thumbnail_url: string
- generation_status: enum (pending, processing, completed, failed)
- created_at: timestamp
- updated_at: timestamp

4. **Message**

- id: UUID (PK)
- conversation_id: UUID (FK to Conversation)
- sender_id: UUID (FK to User)
- body: text
- read_at: timestamp (nullable)
- created_at: timestamp

5. **Project**

- id: UUID (PK)
- owner_id: UUID (FK to User)
- title: string
- description: text
- visibility: enum (private, public, organization)
- created_at: timestamp

    - updated_at: timestamp

6. **ProjectTask**
    - id: UUID (PK)
    - project_id: UUID (FK to Project)
    - title: string
    - status: enum (todo, in_progress, review, done)
    - assignee_id: UUID (FK to User, nullable)
    - due_at: timestamp (nullable)
    - created_at: timestamp
    - updated_at: timestamp

7. **ModerationReport**
    - id: UUID (PK)
    - reporter_id: UUID (FK to User)
    - target_type: enum (post, reel, comment, user, message)
    - target_id: UUID
    - reason: string
    - status: enum (pending, reviewed, action_taken, dismissed)
    - reviewer_id: UUID (FK to User, nullable)
    - created_at: timestamp
    - updated_at: timestamp

Relationships

- User 1:∞ Post (author)
- User 1:∞ Reel (author)
- Post 0:1 Reel (source)
- User 1:∞ Message (sender)
- Conversation 1:∞ Message
- User 1:∞ Project (owner)
- Project 1:∞ ProjectTask
- User 1:∞ ModerationReport (reporter)
- User 0:∞ ModerationReport (reviewer)
- Various entities 0:1 ModerationReport (target)

API Design

RESTful Endpoints

Authentication

```
POST /api/auth/register     # User registration with OTP
POST /api/auth/verify-otp   # OTP verification
POST /api/auth/login        # User login
POST /api/auth/logout       # User logout
GET  /api/auth/me           # Get current user
```

Posts

```
GET    /api/posts           # Get posts (with filtering/paginatio
POST   /api/posts           # Create new post
GET    /api/posts/:id       # Get specific post
PUT    /api/posts/:id       # Update post
DELETE /api/posts/:id       # Delete post
POST   /api/posts/:id/media # Upload media for post
```

Reels

```
GET    /api/reels           # Get reels (with filtering/paginatio
POST   /api/reels           # Create new reel
GET    /api/reels/:id       # Get specific reel
```

Media

```
POST   /api/media           # Upload and process media
```

Messages

```
GET    /api/messages        # Get conversations
GET    /api/messages/:conv  # Get messages in conversation
```

```
POST   /api/messages        # Send new message
```

Projects

```
GET    /api/projects        # Get projects (with filtering)
POST   /api/projects        # Create new project
GET    /api/projects/:id    # Get specific project
PUT    /api/projects/:id    # Update project
DELETE /api/projects/:id    # Delete project
```

Project Tasks

```
GET    /api/projects/:id/tasks     # Get project tasks
POST   /api/projects/:id/tasks     # Create project task
GET    /api/projects/:id/tasks/:t  # Get specific task
PUT    /api/projects/:id/tasks/:t  # Update task
DELETE /api/projects/:id/tasks/:t  # Delete task
```

AI Assistance

```
POST   /api/ai                # AI chat/query endpoint
```

Moderation

```
GET    /api/moderation        # Get moderation queue
POST   /api/moderation        # Submit report
PUT    /api/moderation/:id    # Update report status
```

WebSocket Events

```
connection          # Client connects
disconnect          # Client disconnects
join_room           # Join a room (conversation, project, etc.)
leave_room          # Leave a room
message             # Send chat message
```

```
typing_start       # User starts typing
typing_stop        # User stops typing
notification       # Server sends notification
reel_update        # Reel generation progress
project_update     # Project update notification
```

Security Architecture

Authentication & Authorization

- **JWT Tokens**: Short-lived access tokens (15min) with refresh tokens
- **Password Security**: Argon2id with configurable memory and time costs
- **OTP Verification**: Time-based one-time passwords for registration
- **Role-Based Access Control**: Fine-grained permissions per role
- **Resource Ownership**: Users can only modify their own resources unless granted explicit permissions
- **Admin Functions**: Protected endpoints requiring admin role
- **Moderation Workflow**: Separate role for content moderation

Data Protection

- **Encryption in Transit**: TLS 1.3 for all service communications
- **Encryption at Rest**:
  - Database encryption at storage level (AWS RDS encryption or PostgreSQL TDE)
  - File encryption for sensitive data in object storage
  - Environment variable encryption (Kubernetes Secrets or AWS Secrets Manager)
- **Secrets Management**: No hardcoded credentials; all via secure secret stores
- **Input Validation**: Comprehensive validation using Zod schemas
- **Output Encoding**: Context-aware encoding to prevent XSS
- **SQL Injection Prevention**: Parameterized queries via pg library
- **NoSQl Injection Prevention**: Input validation and sanitization
- **Command Injection Prevention**: Avoid shell commands; use library APIs when necessary

Network Security

- **Zero Trust Networking**: Service-to-service authentication and authorization
- **Network Segmentation**: Separate namespaces for different trust levels
- **Firewall Rules**: Strict ingress/egress controls
- **DDoS Protection**: Rate limiting at API gateway and cloud provider level
- **WAF**: Web Application Firefilter for common web vulnerabilities
- **API Security**: Rate limiting, request size limits, timeout protection

Application Security

- **Dependency Scanning**: Regular vulnerability checks (npm audit, safety)
- **Static Analysis**: ESLint with security plugins, Bandit for Python
- **Dynamic Analysis**: OWASP ZAP scans in CI/CD
- **Dependency Management**: Locked versions with regular updates
- **Privilege Minimization**: Least privilege principle for all services
- **Container Security**: Non-root users, read-only filesystems where possible
- **Immutable Infrastructure**: Infrastructure as Code with Terraform

Performance Optimization

Frontend Optimizations

- **Code Splitting**: Route-based and component-based splitting
- **Lazy Loading**: Images, components, and data loaded on demand
- **Caching Strategy**:
    - HTTP caching with proper cache-control headers
    - SWR/stale-while-revalidate for data fetching
    - Service worker caching for offline capabilities
- **Bundle Optimization**:
    - Tree shaking to remove unused code
    - CSS purging with Tailwind
    - Image optimization with Next.js Image
    - Font optimization and preloading
- **Rendering Strategy**:
    - Server-side rendering for SEO-critical pages
    - Static site generation where appropriate
    - Client-side rendering for dynamic dashboards

- **Performance Budgets**:
  - First Contentful Paint < 1.5s
  - Time to Interactive < 3s
  - Largest Contentful Paint < 2.5s
  - Cumulative Layout Shift < 0.1

Backend Optimizations

- **Database Optimization**:
  - Connection pooling (pgPool for PostgreSQL)
  - Read replicas for distributing read load
  - Query optimization with EXPLAIN ANALYZE
  - Proper indexing strategy (BTREE, GIN, GiST as appropriate)
  - Regular vacuum and analyze operations
  - Partitioning for large tables (time-based for logs/events)
- **Caching Strategy**:
  - Redis for session storage and frequent lookups
  - Multi-level caching (L1: application memory, L2: Redis)
  - Cache warming for predictable access patterns
  - Cache invalidation strategies (time-based, event-based)
  - Distributed locking to prevent cache stampede
- **Async Processing**:
  - BullMQ for background job processing
  - Dead letter queues for failed jobs
  - Retry mechanisms with exponential backoff
  - Priority queues for urgent tasks
  - Rate limiting for external API calls
- **API Optimization**:
  - Pagination for large result sets
  - Field selection to reduce payload size
  - ETag headers for conditional requests
  - Compression (gzip/brotli) for responses
  - HTTP/2 support where available

    - Connection keep-alive and pooling

Infrastructure Optimization

- **Container Resource Management**:
    - CPU and memory requests/limits based on profiling
    - Vertical Pod Autoscaler for right-sizing
    - Node affinity for workload placement
    - Taints and tolerations for specialized hardware
- **Storage Optimization**:
    - SSD-backed storage for databases
    - Object storage lifecycle policies
    - CDN for static asset delivery
    - Edge computing for latency-sensitive operations
- **Network Optimization**:
    - Regional deployment for user proximity
    - Connection pooling for external services
    - Keep-alive connections for database pools
    - HTTP/2 or gRPC for service-to-service communication
    - Compression for inter-service communication

Reliability & Fault Tolerance

Resilience Patterns

- **Circuit Breaker**: Prevent cascading failures (using opossum or similar)
- **Retry Logic**: Exponential backoff with jitter
- **Bulkhead Pattern**: Isolate critical resources
- **Rate Limiting**: Protect against traffic spikes
- **Timeouts**: Configurable timeouts for all external calls
- **Fallback Mechanisms**: Degraded functionality when services unavailable
- **Health Checks**: Liveness and readiness probes for all services

Data Durability

- **Database Backup Strategy**:
    - Automated daily snapshots
    - Point-in-time recovery enabled
    - Cross-region replication for disaster recovery
    - Regular restore testing
- **Object Storage Durability**:
    - Cross-region replication (AWS S3 Cross-Region Replication)
    - Versioning enabled for all buckets
    - Lifecycle policies for cost optimization
    - Object lock for compliance requirements (if needed)
- **Message Queue Persistence**:
    - RabbitMQ/Redeis persistence for critical queues
    - Dead letter queues for failed message analysis
    - Message TTL to prevent queue overflow

Disaster Recovery

- **Multi-Region Deployment**:
    - Active-passive or active-active setup
    - Database replication across regions
    - DNS failover with health checks
    - Automated failover procedures
- **Backup and Restore**:
    - Regular automated backups tested quarterly
    - Runbook documentation for recovery procedures
    - Recovery Time Objective (RTO) < 4 hours
    - Recovery Point Objective (RPO) < 1 hour

Monitoring & Alerting

- **Health Checks**:
    - Liveness probes: Is the application running?
    - Readiness probes: Is the application ready to serve traffic?
    - Startup probes: Has the application finished starting?
- **Service Level Objectives (SLOs)**:
    - Availability: 99.9% monthly uptime
    - Latency: 95% of requests < 200ms
    - Error Rate: < 0.5% error rate
    - Throughput: Handle expected peak load
- **Alerting Strategy**:
    - Critical: Page-ready alerts for system-down scenarios
    - Warning: Ticket-generating alerts for performance degradation
    - Info: Log-only notifications for trends and awareness
    - Alert fatigue prevention through proper threshold tuning

Development Practices

Code Quality

- **Linting**: ESLint (JavaScript/TypeScript), Flake8 (Python)
- **Formatting**: Prettier (JS/TS), Black (Python)
- **Type Checking**: TypeScript strict mode, MyPy (Python)
- **Code Reviews**: Mandatory pull request reviews
- **Branch Strategy**: Trunk-based development with short-lived feature branches
- **Commit Standards**: Conventional commits for changelog generation

Testing Strategy

- **Unit Testing**:
    - Jest for JavaScript/TypeScript
    - Pytest for Python
    - Target: 80%+ line coverage
    - Mock external dependencies

- **Integration Testing**:
    - SuperTest for API endpoint testing
    - Database fixtures for consistent state
    - Service contract testing
- **End-to-End Testing**:
    - Cypress for user flow testing
    - Critical path coverage
    - Cross-browser testing
- **Performance Testing**:
    - k6 or JMeter for load testing
    - Stress testing for breaking point determination
    - Soak testing for memory leak detection
- **Security Testing**:
    - SAST: SonarQube or similar
    - DAST: OWASP ZAP in CI pipeline
    - Dependency scanning: npm audit, safety, dependabot
    - Container scanning: Trivy or Clair

Documentation

- **API Documentation**: Swagger/OpenAPI with automatic generation
- **Architecture Decision Records (ADR)**: For significant architectural decisions
- **Runbooks**: For operational procedures and incident response
- **User Guides**: End-user documentation for platform features
- **Developer Guides**: Onboarding and development setup instructions

Compliance & Governance

Data Protection

- **GDPR Compliance**:

    - Right to access, rectification, erasure
    - Data portability provisions
    - Privacy by design and default
    - Data protection impact assessments

- **Data Retention**:
    - Configurable retention policies per data type
    - Automated deletion of expired data
    - Audit trail preservation requirements

Accessibility

- **WCAG 2.1 AA Compliance**:
    - Keyboard navigation support
    - Screen reader compatibility
    - Color contrast ratios
    - Alternative text for media
    - Form labeling and error identification

Audit & Logging

- **Immutable Logs**: Write-once storage for audit trails
- **Log Retention**: Configurable based on regulatory requirements
- **Log Integrity**: Hash chaining or write-once storage
- **Access Logging**: Who accessed what and when
- **Change Tracking**: Field-level auditing for sensitive data

Future Enhancements & Roadmap

Phase 1: Foundation (Current)

- Core social networking features
- Basic AI assistance
- Document sharing and discussion
- Project collaboration workspaces
- Reel generation from text content

Phase 2: Enhancement

- Advanced AI features (research summarization, literature review assistance)
- Enhanced multimedia support (interactive datasets, 3D models)
- Improved discovery algorithms (personalized recommendations)
- Mobile applications (iOS/Android)
- Offline capabilities with sync

Phase 3: Enterprise

- Single Sign-On (SAML/OAuth)
- Advanced analytics and reporting
- Custom branding and white-labeling
- Advanced compliance features (HIPAA, GDPR automation)
- Dedicated support and SLAs

Phase 4: Innovation

- Real-time collaboration (multi-user editing)
- Augmented reality research visualization
- Blockchain-based provenance tracking
- Advanced predictive analytics for research trends
- Integration with academic repositories (arXiv, PubMed, etc.)

Technical Specifications

Supported Technologies

- **Frontend**: Next.js 16.2.2, React 18, TypeScript, Tailwind CSS
- **Backend**: Node.js 18+, Express.js, PostgreSQL 15, Redis 7
- **AI Services**: Google Gemini API, Qdrant Vector Database
- **Media Processing**: FFmpeg, ImageMagick/sharp
- **Deployment**: Docker, Kubernetes/EKS, Helm
- **Monitoring**: Prometheus, Grafana, ELK stack, Jaeger
- **Testing**: Jest, Pytest, Cypress

- **CI/CD**: GitHub Actions, Terraform

Supported Browsers

- Chrome: Latest 2 versions
- Firefox: Latest 2 versions
- Safari: Latest 2 versions
- Edge: Latest 2 versions
- Mobile Safari: Latest 2 versions
- Chrome Android: Latest 2 versions

Database Schema Versioning

- Migration tool: node-pg-migrate
- Backward compatible changes only
- Data migration scripts for breaking changes
- Rollback procedures for failed migrations

API Versioning

- Version in URL path: `/api/v1/resource/`
- Deprecation policy: 6-month notice for breaking changes
- Semantic versioning for API contract
- Backward compatibility maintained within major version

Environmental Requirements

Development

- Node.js >= 18.0.0
- Python >= 3.9.0
- Docker >= 20.10.0
- Docker Compose >= 2.0.0
- Git >= 2.30.0

Production

- Kubernetes >= 1.24.0
- Helm >= 3.0.0
- PostgreSQL >= 15.0
- Redis >= 7.0
- Object storage (S3-compatible)

Licensing & Dependencies

- **Open Source Components**: MIT, Apache 2.0, ISC licenses predominant
- **Commercial Licenses**: None required for core functionality
- **Managed Services**: AWS/GCP/Azure services used where appropriate
- **Distribution**: Source available under appropriate open source license